

## Introduction to JavaScript

JavaScript is a **high-level, interpreted programming language** that is mainly used to make web pages **interactive and dynamic**.

It is one of the **core technologies of web development**, along with **HTML** and **CSS**.

- **HTML** – defines the structure of a webpage
- **CSS** – defines the style and layout
- **JavaScript (JS)** – adds interactivity and dynamic behavior

When you perform actions like clicking a button, submitting a form, or seeing animations on a website — JavaScript is what makes those interactions happen.

*JavaScript is the **brain of the webpage** — it makes static pages come alive and respond to user actions.*

## Role of JavaScript in Frontend Development

In frontend development, JavaScript:

1. Controls the **behavior** of web pages.
2. Communicates with **HTML and CSS** to make dynamic updates.
3. Enables **real-time user interactions** (like pop-ups, sliders, and form validation).
4. Helps fetch and display **data from APIs** without reloading the page.
5. Supports **modern frameworks** such as **React**, **Angular**, and **Vue** for building Single Page Applications (SPAs).

## Key Features of JavaScript in Frontend Development

| Feature                     | Description                                           | Example / Use                                             |
|-----------------------------|-------------------------------------------------------|-----------------------------------------------------------|
| <b>Lightweight and Fast</b> | Executes directly in the browser without compilation. | Page loads quickly with instant response to user actions. |

| Feature                                | Description                                             | Example / Use                                     |
|----------------------------------------|---------------------------------------------------------|---------------------------------------------------|
| <b>Interpreted Language</b>            | Runs line-by-line in the browser's JavaScript engine.   | No need for separate compilation steps.           |
| <b>Event-Driven</b>                    | Responds to user events like click, hover, scroll, etc. | Button clicks, mouse movements trigger functions. |
| <b>Dynamic HTML Manipulation (DOM)</b> | Can add, remove, or change HTML content dynamically.    | Update page content without reloading.            |
| <b>Platform Independent</b>            | Works in all browsers and operating systems.            | Runs on Chrome, Edge, Firefox, Safari, etc.       |
| <b>Browser Support</b>                 | All modern browsers have built-in JavaScript engines.   | No external installation needed.                  |
| <b>Integration with HTML/CSS</b>       | Easily works with markup and style languages.           | Changes page structure or style dynamically.      |
| <b>Frameworks and Libraries</b>        | Rich ecosystem like React, Angular, Vue, jQuery.        | Makes development faster and easier.              |

## 2. JavaScript Syntax and Basics

### ➤ Comments

// Single-line comment

/\* Multi-line comment \*/

### ➤ Variables

Used to store data values.

var name = "Divya"; // function scoped

let age = 25; // block scoped

```
const city = "Chennai"; // constant
```

### Difference:

| Keyword | Scope    | Reassignment |
|---------|----------|--------------|
| var     | Function | Yes          |
| let     | Block    | Yes          |
| const   | Block    | No           |

## 3. Data Types

| Type      | Example                  | Description                        |
|-----------|--------------------------|------------------------------------|
| String    | "Hello"                  | Text data                          |
| Number    | 42, 3.14                 | Numeric data                       |
| Boolean   | true / false             | Logical values                     |
| Null      | null                     | Empty or no value                  |
| Undefined | undefined                | Variable declared but not assigned |
| Object    | {name: "Divya", age: 22} | Collection of key-value pairs      |
| Array     | [1,2,3]                  | Ordered list                       |

## 4. Operators

### ► Arithmetic Operators

+, -, \*, /, %, \*\*

### ► Assignment Operators

=, +=, -=, \*=, /=

### ► Comparison Operators

==, ===, !=, !==, >, <, >=, <=

## ► Logical Operators

&&, ||, !

## ► Ternary Operator

```
let result = (age > 18) ? "Adult" : "Minor";
```

## 5. Control Flow

### ► If-Else

```
if (marks >= 50) {  
  console.log("Pass");  
} else {  
  console.log("Fail");  
}
```

### ► Switch

```
switch(day) {  
  case 'Mon': console.log("Start of week"); break;  
  case 'Fri': console.log("Weekend soon!"); break;  
  default: console.log("Midweek");  
}
```

## ► Loops

### For Loop

```
for(let i=0; i<5; i++){  
  console.log(i);  
}
```

### While Loop

```
let i = 0;  
while(i < 5){  
  console.log(i);  
  i++;  
}
```

```
}
```

### **Do...While Loop**

```
let j = 0;  
do {  
  console.log(j);  
  j++;  
} while(j < 5);
```

## **6. Functions**

### ► **Function Declaration**

```
function greet(name){  
  return "Hello " + name;  
}
```

### ► **Function Expression**

```
const greet = function(name){  
  return "Hi " + name;  
}
```

### ► **Arrow Function**

```
const greet = (name) => "Welcome " + name;
```

## **7. Objects**

Objects store data as **key-value pairs**.

```
const student = {  
  name: "Divya",  
  age: 21,  
  course: "AI",  
  greet: function(){  
    console.log("Hello " + this.name);  
  }  
}
```

```
};
```

```
student.greet();
```

```
console.log(student.name);
```

## 8. DOM Manipulation (Document Object Model)

DOM allows JavaScript to interact with HTML and CSS.

### Selecting Elements:

```
document.getElementById("demo");
```

```
document.querySelector(".class");
```

```
document.querySelectorAll("p");
```

### Changing Content:

```
document.getElementById("demo").innerHTML = "Hello World!";
```

### Changing Style:

```
document.getElementById("demo").style.color = "blue";
```

## 9. Events

In JavaScript, **events** are **actions or occurrences** that happen in the browser, often triggered by the **user's interaction** or the **browser itself**.

### Example of events:

- A user clicks a button
- A web page finishes loading
- A key is pressed
- A mouse moves over an image
- A form is submitted

JavaScript allows you to **detect** and **respond** to these events — making web pages **interactive** and **dynamic**.

## Types of Events in JavaScript

| Category               | Event Name                                                           | Description                          |
|------------------------|----------------------------------------------------------------------|--------------------------------------|
| <b>Mouse Events</b>    | onclick, ondblclick, onmouseover, onmouseout, onmousedown, onmouseup | Triggered by mouse actions           |
| <b>Keyboard Events</b> | onkeydown, onkeypress, onkeyup                                       | Triggered by keyboard actions        |
| <b>Form Events</b>     | onsubmit, onchange, onfocus, onblur                                  | Triggered by form field interactions |
| <b>Window Events</b>   | onload, onresize, onscroll, onunload                                 | Triggered by browser window actions  |

## Ways to Handle Events

There are **three main methods** to handle events in JavaScript:

### 1. Inline Event Handling (Inside HTML Tag)

```
<button onclick="showMessage()">Click Me</button>
```

```
<script>
```

```
function showMessage() {  
    alert("Button was clicked!");  
}
```

```
</script>
```

✅ **Simple and direct**, but not suitable for large applications.

### 2. Event Property Method (Assigning Event in Script)

```
<button id="btn">Click Here</button>
```

```
<script>
```

```
document.getElementById("btn").onclick = function() {  
    alert("Hello from JS!");  
}
```

```
};  
</script>
```

✔ Keeps JavaScript separate from HTML.

Only one event handler per event can be assigned this way.

## Commonly Used Events with Examples

### ► Mouse Events

```
<button onmouseover="changeColor()" onmouseout="resetColor()"
id="myBtn">Hover Me</button>
<script>
function changeColor(){
    document.getElementById("myBtn").style.backgroundColor = "lightgreen";
}
function resetColor(){
    document.getElementById("myBtn").style.backgroundColor = "";
}
</script>
```

### ► Keyboard Events

```
<input type="text" onkeydown="keyPressed()" placeholder="Type something...">
<script>
function keyPressed(){
    console.log("Key pressed!");
}
</script>
```

### ► Form Events

```
<form onsubmit="return validateForm()">
    <input type="text" id="name" placeholder="Enter name">
    <input type="submit" value="Submit">
```



```
</form>
```

```
<script>
```

```
function validateForm(){
```

```
    let name = document.getElementById("name").value;
```

```
    if(name === ""){
```

```
        alert("Name cannot be empty!");
```

```
        return false; // prevent form submission
```

```
    }
```

```
    return true;
```

```
}
```

```
</script>
```

## ► Window Events

```
<body onload="pageLoaded()" onresize="windowResized()">
```

```
<script>
```

```
function pageLoaded(){
```

```
    console.log("Page is loaded successfully!");
```

```
}
```

```
function windowResized(){
```

```
    console.log("Window resized!");
```

```
}
```

```
</script>
```

## Event Object

Every event in JavaScript has an **event object** that contains details about the event (like which key was pressed or which element was clicked).

Example:

```
document.addEventListener("click", function(event){
```

```
console.log("You clicked at X:", event.clientX, "Y:", event.clientY);  
});
```

## Navigation in Web Development

**Navigation** in web development means **moving between different parts of a website** — such as from one page to another or from one section to another **smoothly and easily**.

It helps users explore a website and access the information they need quickly.

Good navigation improves **user experience (UX)** and makes a website **interactive and organized**.

### Types of Navigation in Web Development

| Type                                                 | Description                                                                    | Example / Use Case                                         |
|------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>1. Page-to-Page Navigation</b>                    | Moving between different HTML pages.                                           | Clicking a link to open about.html from index.html.        |
| <b>2. Section Navigation</b>                         | Jumping to specific sections on the same page.                                 | Clicking “Go to Top” or “Read More” within a long webpage. |
| <b>3. Programmatic Navigation (JavaScript-based)</b> | Controlled using JavaScript functions or browser APIs.                         | Buttons that redirect using window.location.href.          |
| <b>4. Single Page Application (SPA) Navigation</b>   | Uses frameworks like React/Angular — changes views without reloading the page. | Smooth transitions in dynamic web apps.                    |

## 1. HTML Navigation using <a> Tag

The <a> (anchor) tag is the **basic navigation element** in HTML.

### ► Syntax:

```
<a href="page.html">Go to Page</a>
```

### Attributes:

- href → The destination (file name, URL, or section ID)
- target="\_blank" → Opens link in a new tab
- id → Used for internal navigation (anchor links)

### ► Example (Page-to-Page Navigation):

```
<a href="about.html">About Us</a>
```

```
<a href="contact.html" target="_blank">Contact Us</a>
```

## 2. Section Navigation (Same Page)

You can use **ID-based links** to jump to specific parts of the same page.

### ► Example:

```
<a href="#services">Go to Services</a>
```

```
<h2 id="services">Our Services</h2>
```

```
<p>We provide web design, app development, and more.</p>
```

When you click **“Go to Services”**, it scrolls to the section with id="services".

## 3. Programmatic Navigation using JavaScript

JavaScript can also be used to navigate between pages **dynamically**.

### ► Example 1: Go to Another Page

```
<button onclick="goToAbout()">Go to About Page</button>
```

```
<script>
```

```
function goToAbout(){
```

```
    window.location.href = "about.html";
```

```
}
```

```
</script>
```

► **Example 2: Go Back or Forward in Browser History**

```
<button onclick="goBack()">Go Back</button>
```

```
<button onclick="goForward()">Go Forward</button>
```

```
<script>
```

```
function goBack(){
```

```
    window.history.back();
```

```
}
```

```
function goForward(){
```

```
    window.history.forward();
```

```
}
```

```
</script>
```